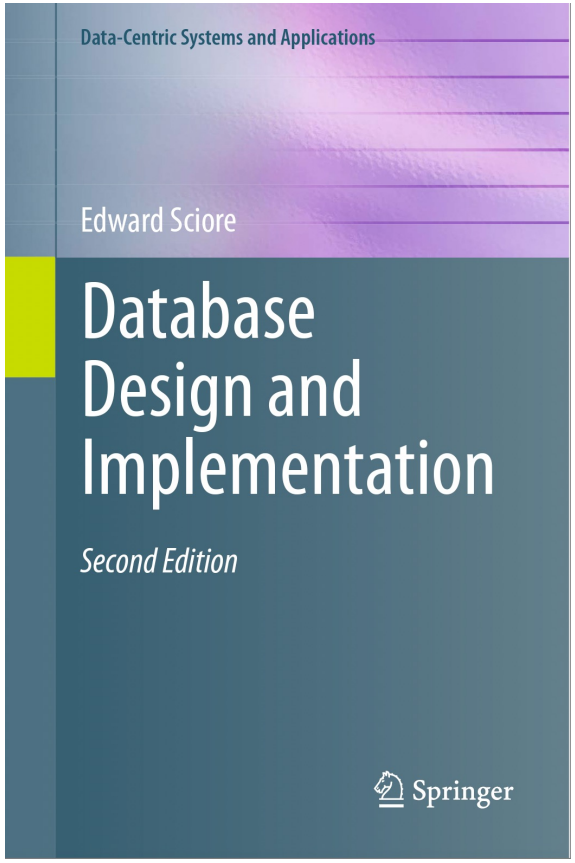
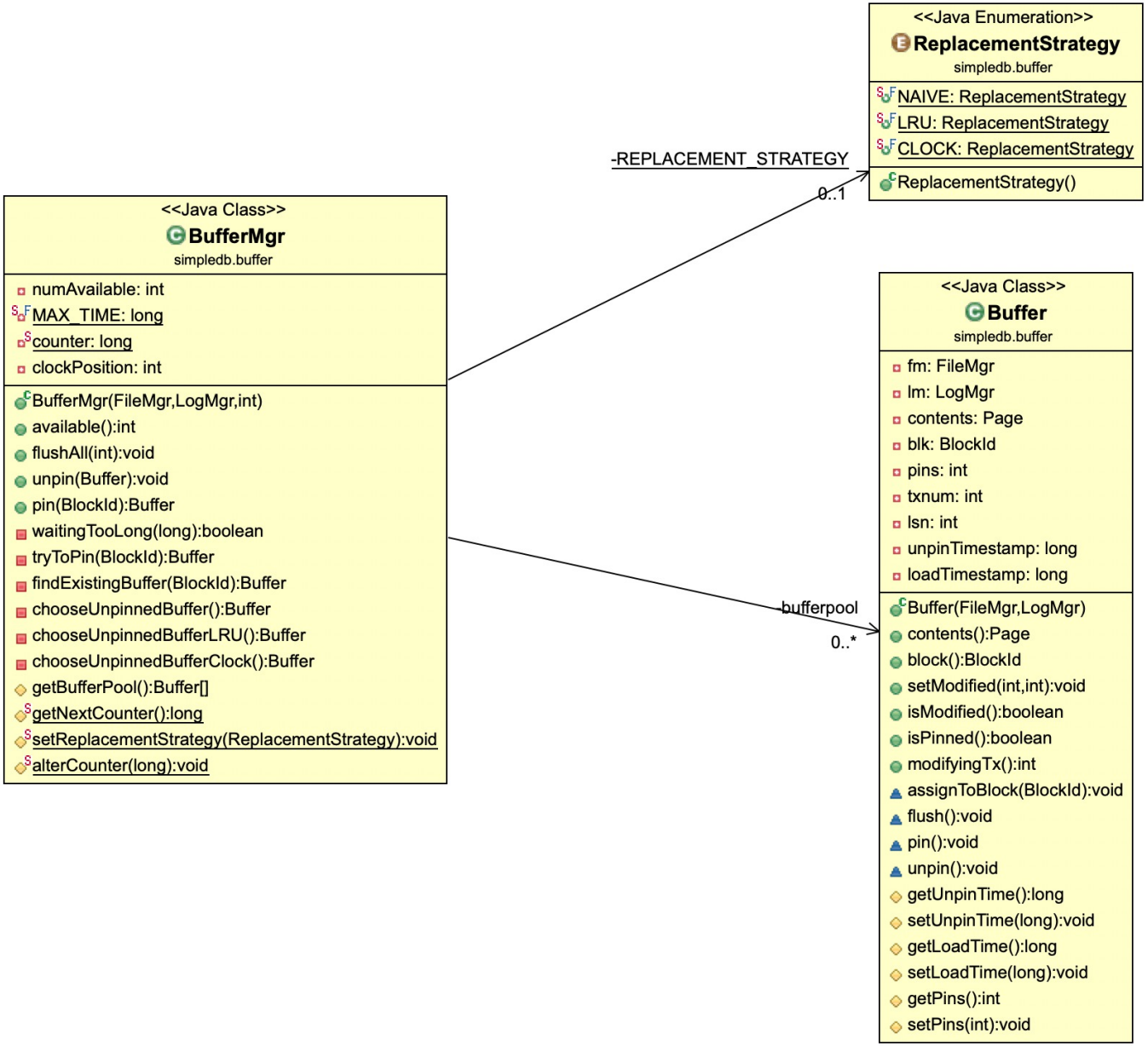
SimpleDB, parte 3

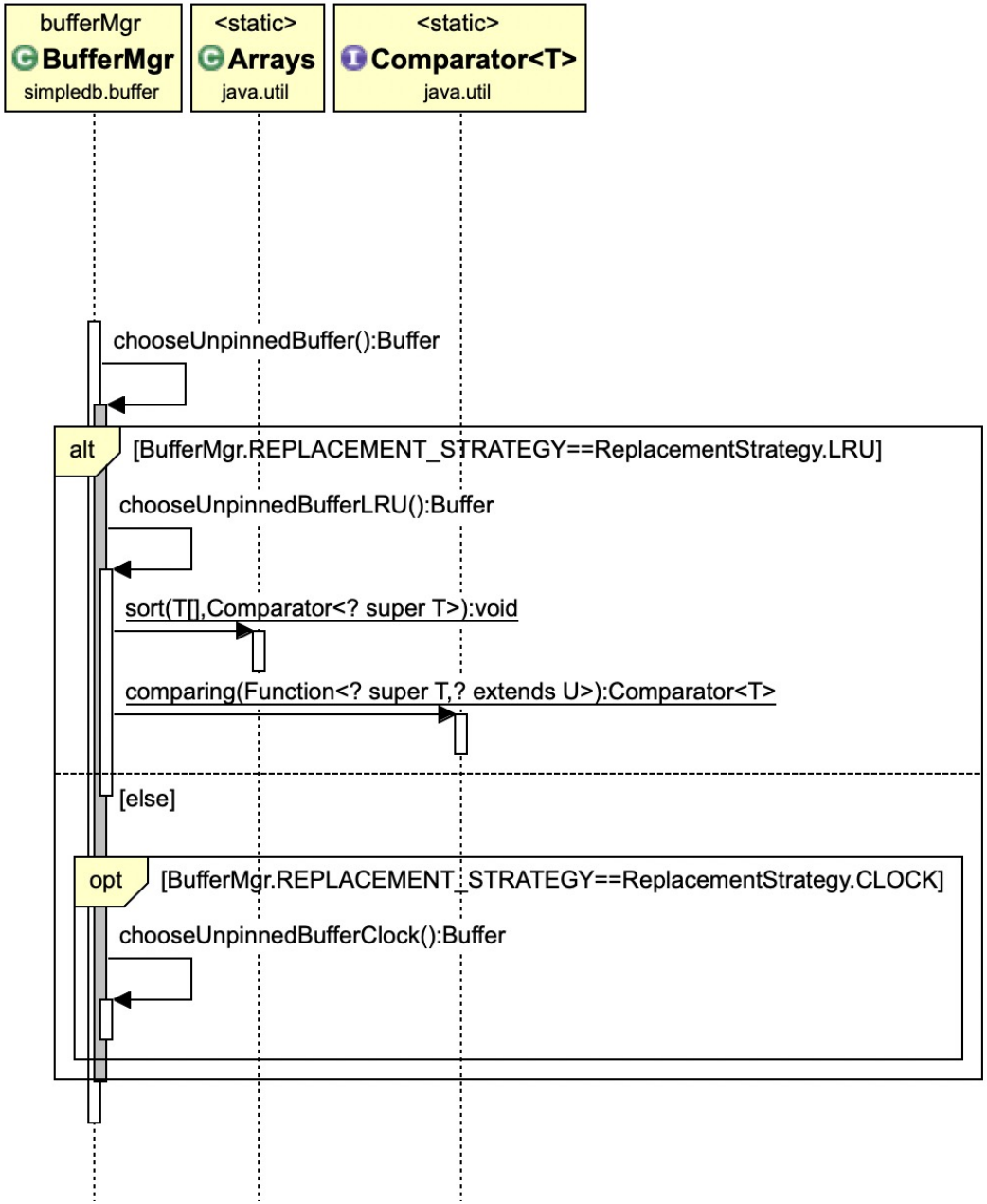
Luigi Bellomarini
luigi.bellomarini@uniroma3.it



Esercizio pratico 2.1

- SimpleDB utilizza la strategia naïve
- Implementare le altre strategie (FIFO, LRU, clock) e renderle attivabili in SimpleDB.
- Considerare l'esercizio d'esame Prova parziale del 28/03/2012, domanda 4 già discusso a lezione.
- Implementare una classe di test che:
  - Imposta lo stato del buffer pool descritto nella prova parziale
  - Esegue tutte le operazioni di modifica del buffer descritte attivando le varie strategie
  - Produce in output la tabella di informazioni richiesta dall'esercizio.


<<Java Enumeration>>
ReplacementStrategy
simpledb.buffer
NAIVE: ReplacementStrategy
LRU: ReplacementStrategy
CLOCK: ReplacementStrategy
ReplacementStrategy()
-REPLACEMENT_STRATEGY
0..1
<<Java Class>>
BufferMgr
simpledb.buffer
numAvailable: int
MAX_TIME: long
counter: long
clockPosition: int
BufferMgr(FileMgr,LogMgr,int)
available():int
flushAll(int):void
unpin(Buffer):void
pin(BlockId):Buffer
waitingTooLong(long):boolean
tryToPin(BlockId):Buffer
findExistingBuffer(BlockId):Buffer
chooseUnpinnedBuffer():Buffer
chooseUnpinnedBufferLRU():Buffer
chooseUnpinnedBufferClock():Buffer
getBufferPool():Buffer[]
getNextCounter():long
setReplacementStrategy(ReplacementStrategy):void
alterCounter(long):void
bufferpool
0..*
<<Java Class>>
Buffer
simpledb.buffer
fm: FileMgr
lm: LogMgr
contents: Page
blk: BlockId
pins: int
txnum: int
lsn: int
unpinTimestamp: long
loadTimestamp: long
Buffer(FileMgr,LogMgr)
contents():Page
block():BlockId
setModified(int,int):void
isModified():boolean
isPinned():boolean
modifyingTx():int
assignToBlock(BlockId):void
flush():void
pin():void
unpin():void
getUnpinTime():long
setUnpinTime(long):void
getLoadTime():long
setLoadTime(long):void
getPins():int
setPins(int):void


bufferMgr
BufferMgr
simpledb.buffer
<static>
Arrays
java.util
<static>
Comparator<T>
java.util
chooseUnpinnedBuffer():Buffer
alt
[BufferMgr.REPLACEMENT_STRATEGY==ReplacementStrategy.LRU]
chooseUnpinnedBufferLRU():Buffer
sort(T[],Comparator<? super T>):void
comparing(Function<? super T,? extends U>):Comparator<T>
[else]
opt
[BufferMgr.REPLACEMENT_STRATEGY==ReplacementStrategy.CLOCK]
chooseUnpinnedBufferClock():Buffer

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

Nella figura seguente è schematizzato un piccolissimo buffer con quattro pagine (numerate da 0 a 3), il cui stato viene descritto, per ciascuna pagina, da (i) un intero che indica il numero di pin su di essa (quindi 0 indica che la pagina è libera) (ii) un riferimento al blocco che per ultimo è stato caricato nella pagina; (iii) l'istante in cui è stato effettuato l'ultimo caricamento; (iv) l'istante in cui la pagina è stata per l'ultima volta liberata (se è libera) (v) un booleano che indica se la pagina è sporca (1 indica che è stata modificata dopo il caricamento o dopo l'ultima scrittura).

Pagina del buffer:	0	1	2	3
numero di pin sulla pagina	1	2	0	1
blocco	70	33	35	47
istante load	1	7	3	9
istante unpin			8	
dirty	1	0	0	0

Si supponga ora che vengano eseguite (a partire dall'istante 10, dopo che all'istante 9 è stata caricata la pagina 3) le seguenti operazioni (in cui l'argomento delle pin e unpin è il blocco di interesse, con setXXX si indica un aggiornamento di un qualche valore nel blocco e con flushAll il flush dell'intero buffer):

Riempiendo la tabella seguente, indicare, per ciascuna delle strategie e per ciascuna delle pin, (1) quale pagina del buffer viene utilizzata, (2) se il blocco corrispondente viene letto dal disco e (3) se viene eseguita una scrittura su disco.

	naif			LRU			clock		
Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
pin(60)									
pin(70)									
pin(60)									
pin(70)									

10	unpin(70)
11	pin(60)
12	setXXX(60,...)
13	unpin(60)
14	flushAll
15	setXXX(47,...)
16	unpin(47)
17	pin(70)
18	setXXX(70,...)
19	unpin(70)
20	pin(60)
21	unpin(60)
22	pin(70)

	0	1	2	3
N. PIN	1	2	0	1
BLOCCO	70	33	35	47
1ST LOAD	1	7	3	9
1ST UNPIN			8	
DIRTY	1	0	0	0

		naif			LRU			clock		
Istante	Operazione	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?	Pagina	Legge?	Scrive?
11	pin(60)	0	SI	SI	2	SI	NO			
17	pin(70)	0	SI	NO	0	NO	NO			
20	pin(60)	0	SI	SI	2	NO	NO			
22	pin(70)	0	SI	NO	0	NO	NO			

SimpleDB, componenti "read-only"

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco

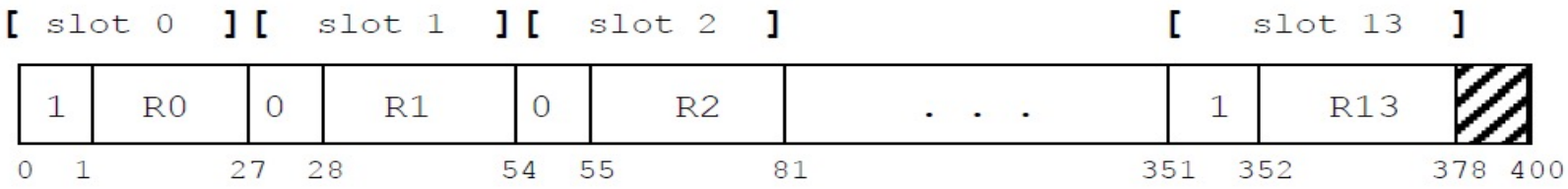
Record Manager

- The portion of the database system that is responsible for giving structure to a file is called the *record manager*
- It organizes a file into a collection of records, and has methods for iterating through the records and placing values in them
- **The record manager is the portion of the database system that is responsible for storing records in a file**
- When we design a record manager, we need to address the following issues:
  1. Should each record be placed entirely within one block?
  2. Will all of the records in a block be from the same table?
  3. Is each field representable using a fixed number of bytes?
  4. Where should each field value in a record be stored?

Record Manager
(SimpleDB standard implementation)

- **straightforward implementation (SimpleDB):**
  - homogeneous, unspanned, and fixed length records
- the records are homogeneous and fixed-length means that we can allocate the same amount of space for each record within a block
- we can think of each **block** as an **array of records**

- **Record Page**: a block that is structured as an array of fixed-length records



- 0 or 1 is used to **flag if the slot is used or not** (delete means turn a 1 to 0)
- Each record in a page has an ID. When records are fixed-length, the ID can be its slot number in the page.

Record Manager (SimpleDB standard implementation)

Record Length: 26
Field Information:

Name	Type	Length	Offset
SId	int	4	0
SName	varchar	14	4
GradYear	int	4	18
MajorId	int	4	22

Figure 15-6: The table information for STUDENT

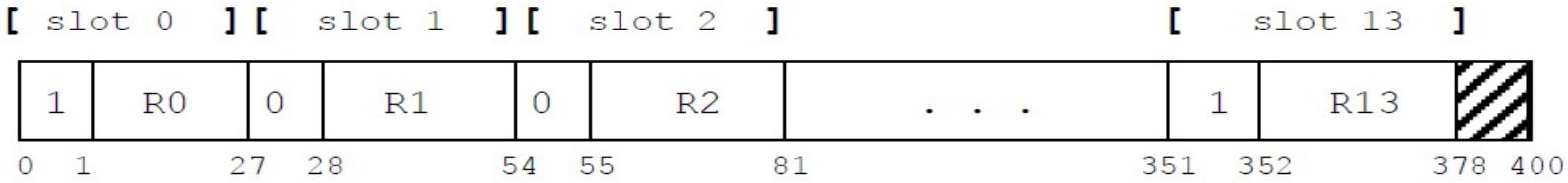
- The record manager needs to be able to **insert**, **delete**, and **modify** records in a record page.
- It uses a **Table Information** for each table, with:
  - the record length;
  - the name, type, length, and offset of each field of a record (**Schema**)

Record Manager (SimpleDB standard implementation)

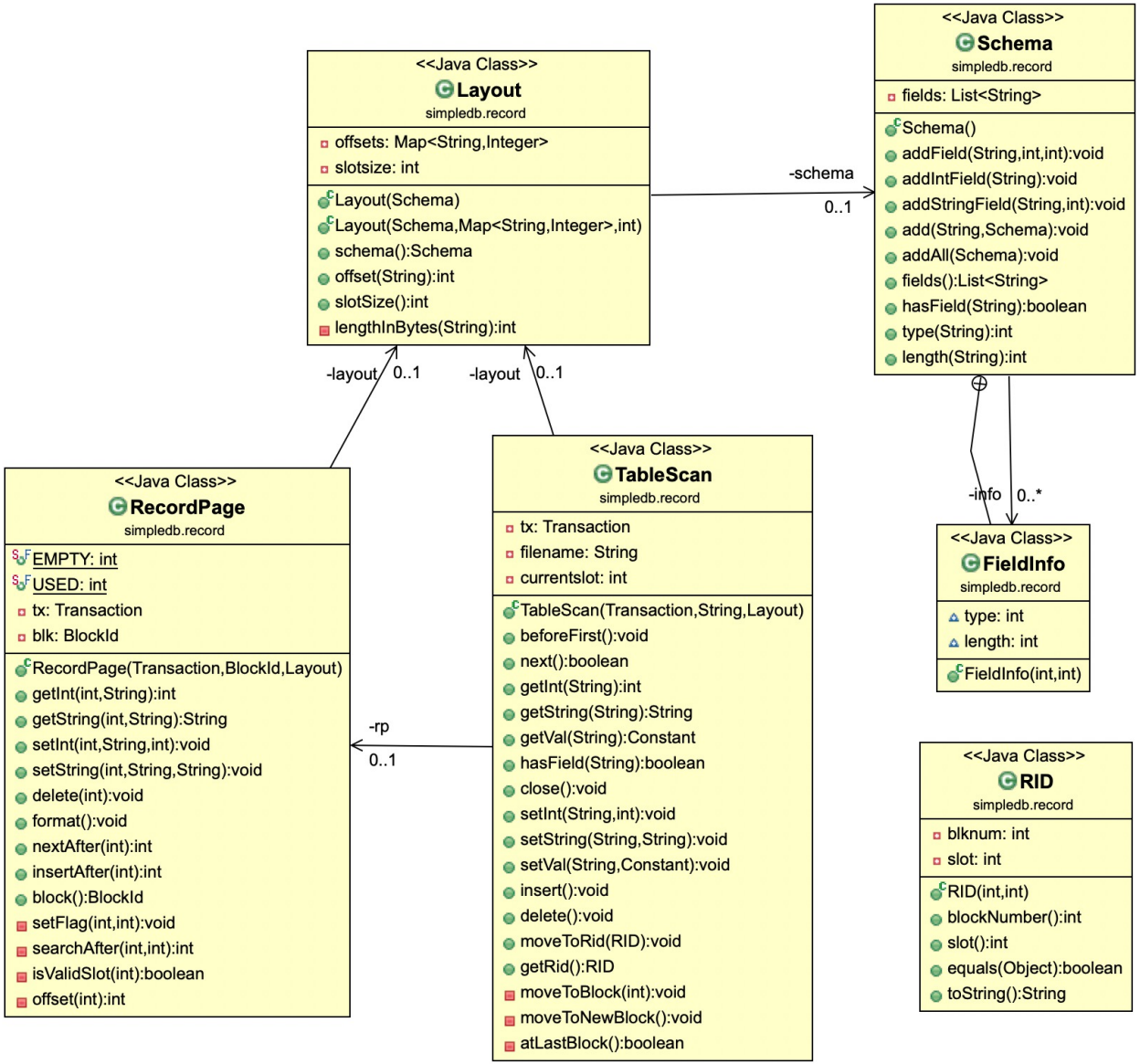
- Let **RL** = **record length**
- Empty/in use flag for slot k
  - (RL+1) * k

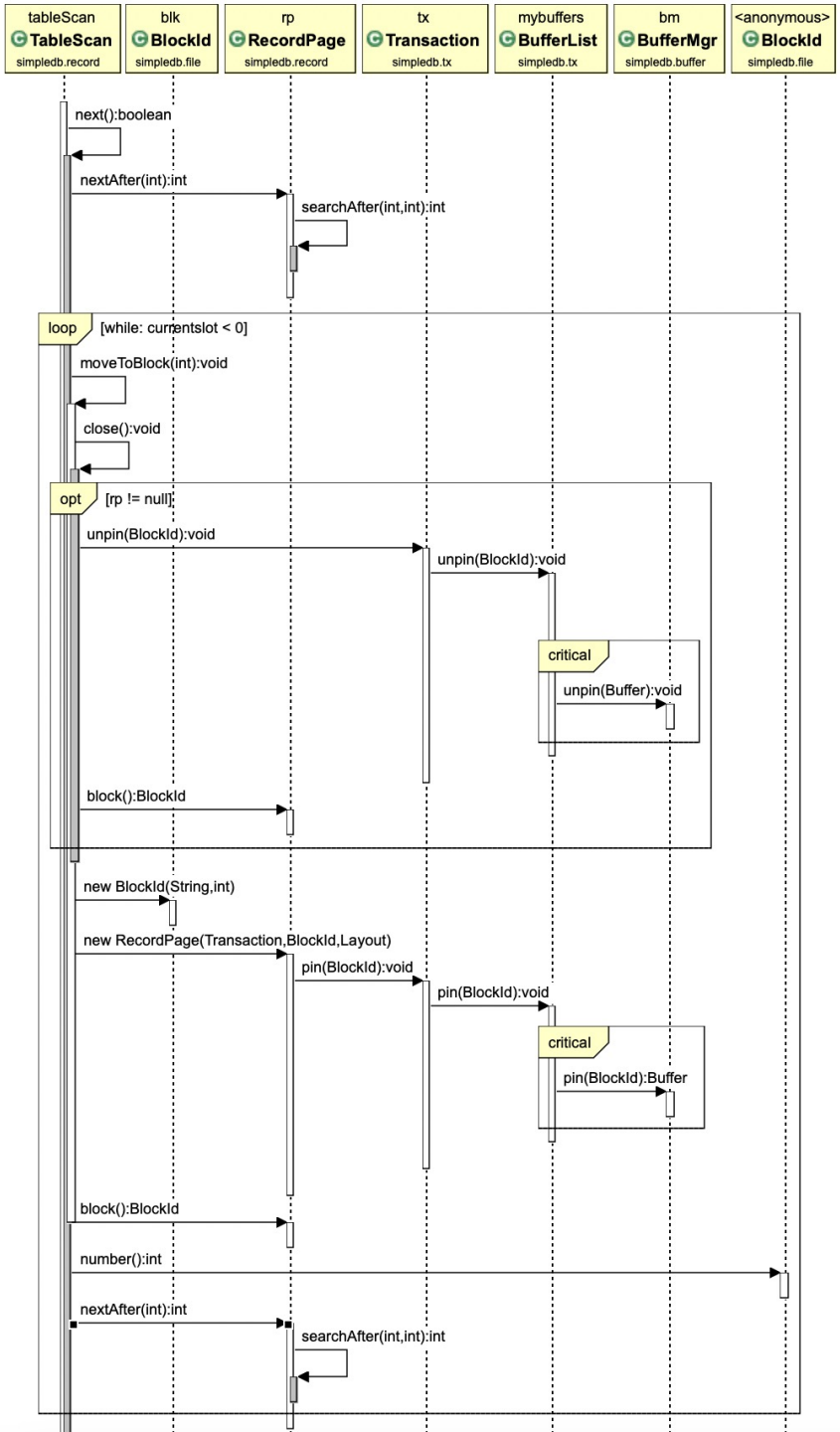
- The value of field F in the record at slot k
  - (RL+1) * k + OFFSET(F) + 1

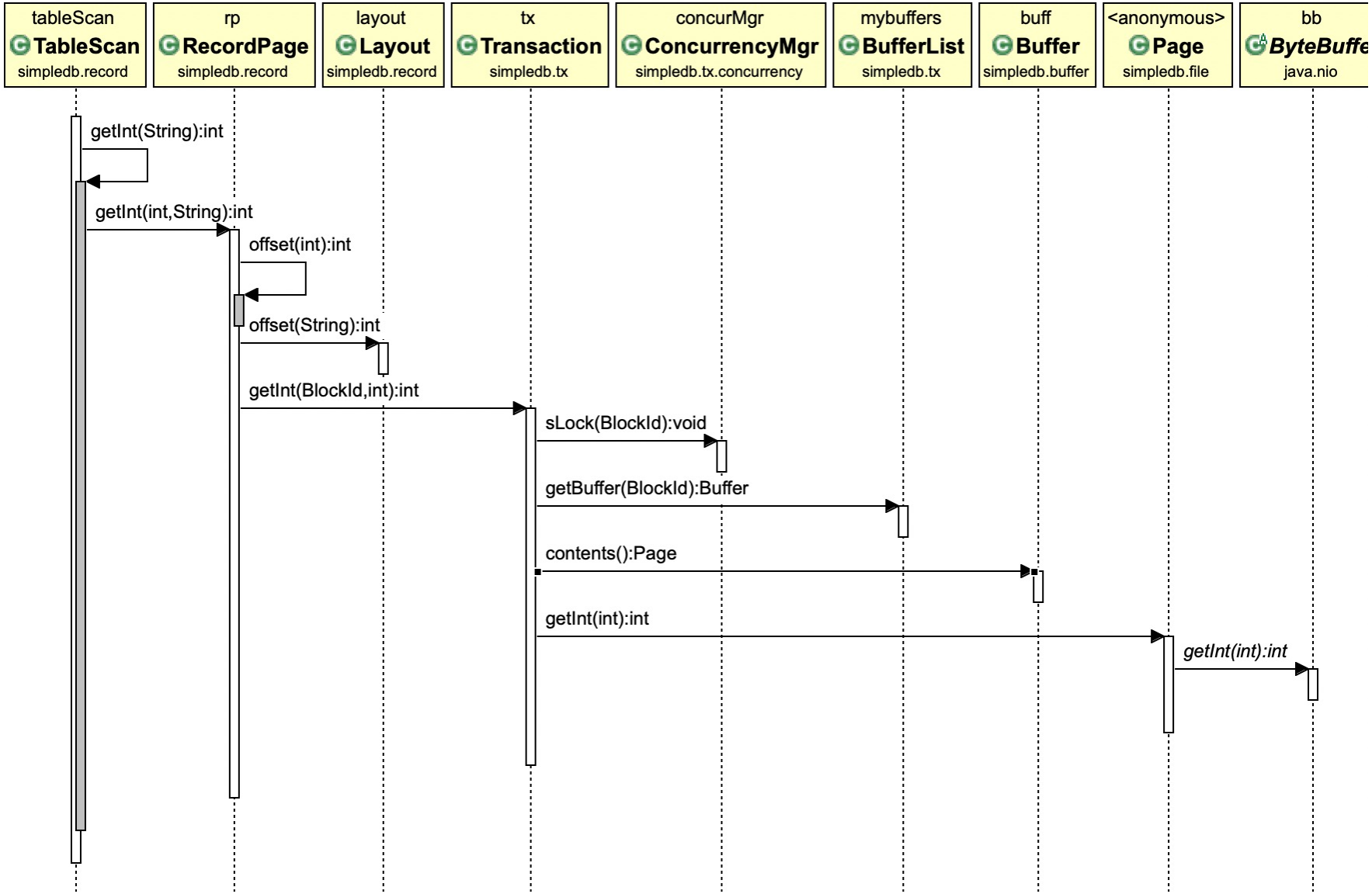
- Inserting, deleting, modifying, retrieving records (the straightforward way…)

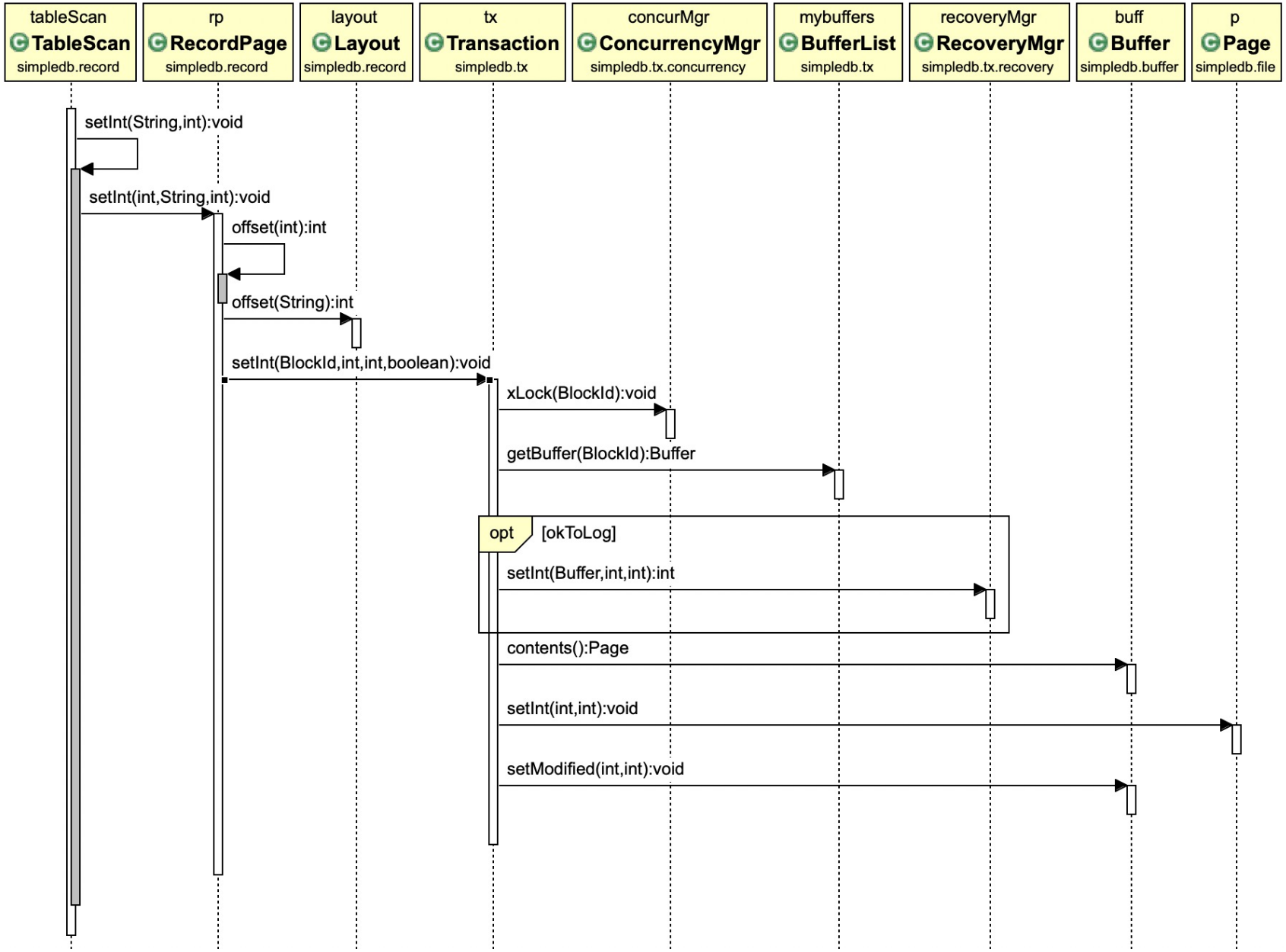


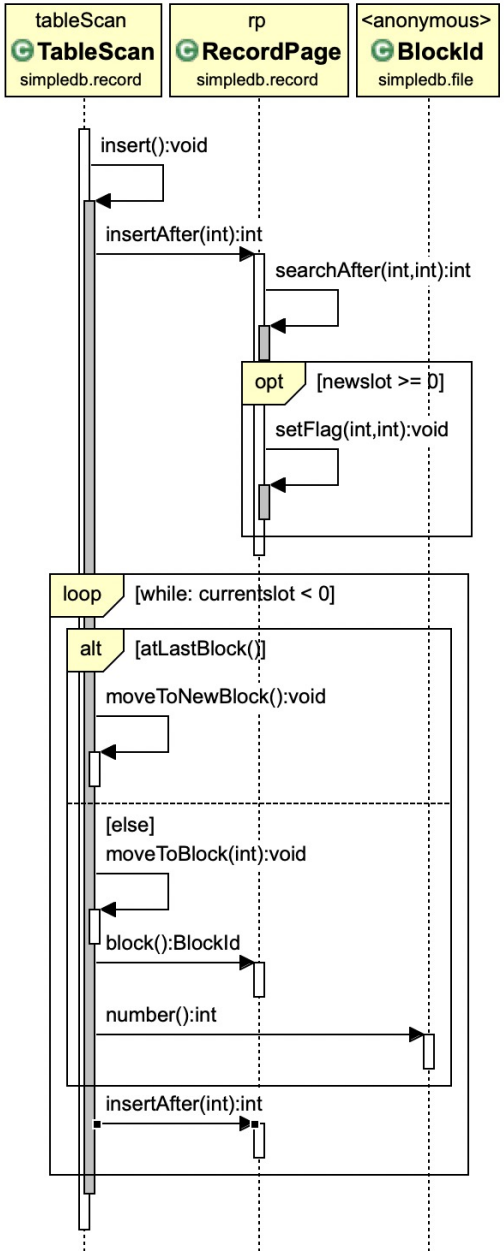
Record Manager, classi




tableScan
TableScan
simpledb.record
blk
BlockId
simpledb.file
rp
RecordPage
simpledb.record
tx
Transaction
simpledb.tx
mybuffers
BufferList
simpledb.tx
bm
BufferMgr
simpledb.buffer
<anonymous>
BlockId
simpledb.file
next():boolean
nextAfter(int):int
searchAfter(int,int):int
loop
[while: currentslot < 0]
moveToBlock(int):void
close():void
opt
[rp != null]
unpin(BlockId):void
unpin(BlockId):void
critical
unpin(Buffer):void
block():BlockId
new BlockId(String,int)
new RecordPage(Transaction,BlockId,Layout)
pin(BlockId):void
pin(BlockId):void
critical
pin(BlockId):Buffer
block():BlockId
number():int
nextAfter(int):int
searchAfter(int,int):int


tableScan
TableScan
simpledb.record
rp
RecordPage
simpledb.record
layout
Layout
simpledb.record
tx
Transaction
simpledb.tx
concurMgr
ConcurrencyMgr
simpledb.tx.concurrency
mybuffers
BufferList
simpledb.tx
buff
Buffer
simpledb.buffer
<anonymous>
Page
simpledb.file
bb
ByteBuffe
java.nio
getInt(String):int
getInt(int,String):int
offset(int):int
offset(String):int
getInt(BlockId,int):int
sLock(BlockId):void
getBuffer(BlockId):Buffer
contents():Page
getInt(int):int
getInt(int):int


tableScan
TableScan
simpledb.record
rp
RecordPage
simpledb.record
layout
Layout
simpledb.record
tx
Transaction
simpledb.tx
concurMgr
ConcurrencyMgr
simpledb.tx.concurrency
mybuffers
BufferList
simpledb.tx
recoveryMgr
RecoveryMgr
simpledb.tx.recovery
buff
Buffer
simpledb.buffer
p
Page
simpledb.file
setInt(String,int):void
setInt(int,String,int):void
offset(int):int
offset(String):int
setInt(BlockId,int,int,boolean):void
xLock(BlockId):void
getBuffer(BlockId):Buffer
opt
[okToLog]
setInt(Buffer,int,int):int
contents():Page
setInt(int,int):void
setModified(int,int):void


tableScan
TableScan
simpledb.record
rp
RecordPage
simpledb.record
<anonymous>
BlockId
simpledb.file
insert():void
insertAfter(int):int
searchAfter(int,int):int
opt
[newslot >= 0]
setFlag(int,int):void
loop
[while: currentslot < 0]
alt
[atLastBlock()]
moveToNewBlock():void
[else]
moveToBlock(int):void
block():BlockId
number():int
insertAfter(int):int

Esercizio pratico 3.1

- Consideriamo il seguente setting (una variante dell'esercizio d'esame Prova parziale del 30/03/2015, domanda 1).
- R è una relazione definita su due campi A (di tipo intero) e B (di tipo stringa), di lunghezza:
  - a = 4 byte; b = 12 caratteri
- R è memorizzata sulla struttura fisica sequenziale di SimpleDB
- Utilizziamo blocchi di 2000 byte e un buffer di 500 blocchi
- Utilizziamo la strategia di rimpiazzo naive.

- Implementare le seguenti operazioni interagendo direttamente con le classi del package **record**:
  - **Inserimento** di 100.000 ennuple con valori progressivi per A a partire da 0 e stringhe casuali per B.
    - Visualizzare il numero di blocchi scritti per creare la relazione
  - **Cancellare** le ennuple per le quali 0<=A<20.000
    - Visualizzare il numero di blocchi cancellati
  - **Inserimento** di 10.000 ennuple con valori casuali di A compresi tra 0 e 100 a partire dal primo blocco
    - Visualizzare il numero di blocchi scritti per effettuare l'inserimento
  - Generare un insieme S di 500 numeri casuali compresi tra 0 e 1000 e **contare il numero delle ennuple** per le quali il valore di A appartiene a S
    - Visualizzare il numero di blocchi letti per effettuare il conteggio.